Air University

Department of Computer Science

Introduction to Parallel & Distributed Computing (CS426L)

Lab Report 08: Distributed-Memory Computing, Message Passing & Distributed-System Architectures

C# Edition (.NET, MPI.NET on MS-MPI)

Student Name	Fahad Noor
Registration No.	232448
Section	BSCS-ev-7-C
Instructor	Muhammad Nauman Zafar Hashmi
Date	Thursday, October 22, 2026

Section 1: Environment Setup

MS-MPI (runtime and SDK) was installed and verified with mpiexec -n 2 hostname. A working folder for Lab 08 was created with three console projects: HelloMpi, SumCompare and Styles. Each project targets .NET Framework 4.8 (net48) and references the MPI.NET library from NuGet; the project file used for all three is shown below. Every program was built with dotnet build -c Release and started with mpiexec -n <ranks>.

```
<Project Sdk="Microsoft.NET.Sdk">

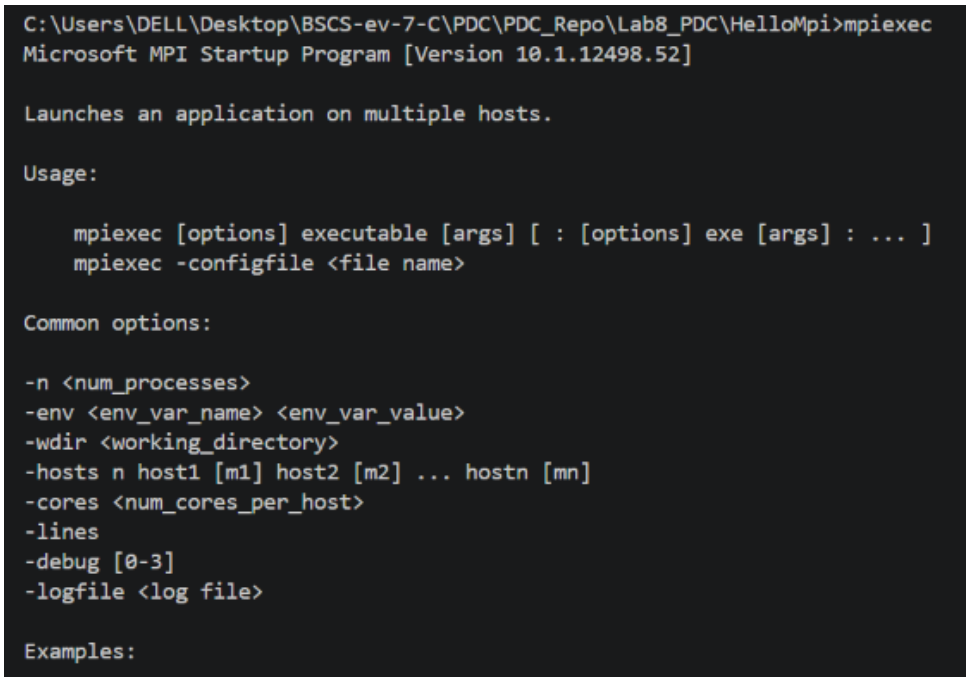
 <PropertyGroup>
  <OutputType>Exe</OutputType>
  <TargetFramework>net48</TargetFramework>
  <LangVersion>latest</LangVersion>
  <PlatformTarget>x64</PlatformTarget>
 </PropertyGroup>

 <ItemGroup>
  <PackageReference Include="MPI.NET" Version="1.2.0" />
 </ItemGroup>

</Project>
```

MS-MPI version: 10.1.12498.52 (Microsoft MPI 10.1)

Logical cores (Environment.ProcessorCount): 8



Section 2: Task 1 — Verify MPI and Run Hello World

Source file: HelloMpi/Program.cs

```
// Lab 08 - Task 1: Verify MPI and Run Hello World
// Every rank prints one line straight away (unordered), then sends the same
// report line to rank 0, which prints all of them in rank order (ordered).
//
// Run:  mpiexec -n 4 .\bin\Release\net48\HelloMpi.exe

using System;
using System.Threading;
using MPI;

class Program
{
    const int ReportTag = 0;
```

```
static void Main(string[] args)
{
    // MPI.Environment starts MPI for this process and shuts it down when disposed.
    using (new MPI.Environment(ref args))
    {
        Intracommunicator comm = Communicator.world;   // MPI_COMM_WORLD: all ranks started by mpiexec

        // ---- query rank, size and processor name ----
        int rank = comm.Rank;                          // this process's ID: 0 .. size-1
        int size = comm.Size;                          // number of processes in the communicator
        string processorName = MPI.Environment.ProcessorName;

        string line = $"Hello from rank {rank} of {size} on {processorName}";

        // ---- part 1: every rank prints on its own (order decided by the OS) ----
        Console.WriteLine($"[unordered] {line}");
        Console.Out.Flush();

        comm.Barrier();                                // all ranks have printed

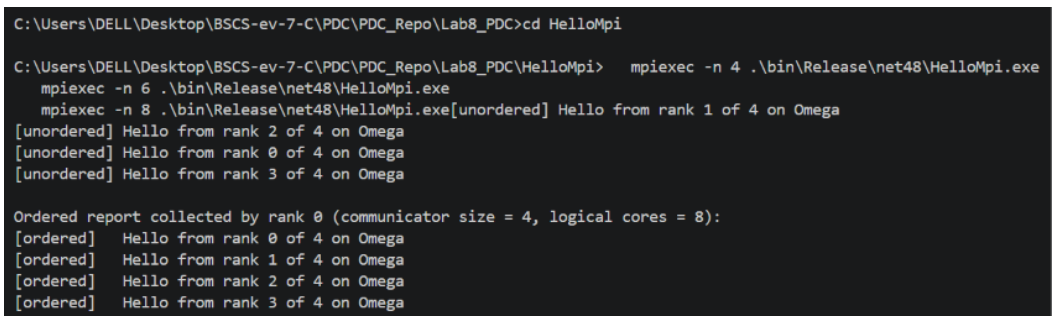
        // ---- part 2: send the report line to rank 0, which prints in rank order ----
        if (rank != 0)
        {
            comm.Send<string>(line, 0, ReportTag);
        }
        else
        {
            Thread.Sleep(300);                         // let the unordered lines reach the console first
            Console.WriteLine();
            Console.WriteLine($"Ordered report collected by rank 0 (communicator size = {size}, " +
                              $"logical cores = {System.Environment.ProcessorCount}):");
            Console.WriteLine($"[ordered]   {line}");
            for (int source = 1; source < size; source++)
            {
                string received = comm.Receive<string>(source, ReportTag);
                Console.WriteLine($"[ordered]   {received}");
            }
        }
    }
}
```

Hello World with different numbers of ranks

Ranks started	Communicator size reported	Order of ranks, run 1	Order of ranks, run 2	Processor name(s)
4	4	1, 0, 2, 3	1, 2, 0, 3	Omega (all ranks)
6	6	1, 0, 4, 2, 3, 5	5, 1, 2, 4, 0, 3	Omega (all ranks)
8	8	1, 0, 2, 5, 4, 6, 7, 3	0, 4, 2, 1, 5, 6, 3, 7	Omega (all ranks)

The two "order" columns give the order in which the unordered lines appeared; the ordered report was in rank order 0, 1, 2, ... in every run.

MS-MPI version: 10.1.12498.52 (Microsoft MPI 10.1) Logical cores (Environment.ProcessorCount): 8



```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\HelloMpi>   mpiexec -n 6 .\bin\Release\net48\HelloMpi.exe
[unordered] Hello from rank 5 of 6 on Omega
[unordered] Hello from rank 1 of 6 on Omega
[unordered] Hello from rank 2 of 6 on Omega
[unordered] Hello from rank 4 of 6 on Omega
[unordered] Hello from rank 0 of 6 on Omega
[unordered] Hello from rank 3 of 6 on Omega

Ordered report collected by rank 0 (communicator size = 6, logical cores = 8):
[ordered]   Hello from rank 0 of 6 on Omega
[ordered]   Hello from rank 1 of 6 on Omega
[ordered]   Hello from rank 2 of 6 on Omega
[ordered]   Hello from rank 3 of 6 on Omega
[ordered]   Hello from rank 4 of 6 on Omega
[ordered]   Hello from rank 5 of 6 on Omega
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\HelloMpi>   mpiexec -n 8 .\bin\Release\net48\HelloMpi.exe
[unordered] Hello from rank 0 of 8 on Omega
[unordered] Hello from rank 4 of 8 on Omega
[unordered] Hello from rank 2 of 8 on Omega
[unordered] Hello from rank 1 of 8 on Omega
[unordered] Hello from rank 5 of 8 on Omega
[unordered] Hello from rank 6 of 8 on Omega
[unordered] Hello from rank 3 of 8 on Omega
[unordered] Hello from rank 7 of 8 on Omega

Ordered report collected by rank 0 (communicator size = 8, logical cores = 8):
[ordered]   Hello from rank 0 of 8 on Omega
[ordered]   Hello from rank 1 of 8 on Omega
[ordered]   Hello from rank 2 of 8 on Omega
[ordered]   Hello from rank 3 of 8 on Omega
[ordered]   Hello from rank 4 of 8 on Omega
[ordered]   Hello from rank 5 of 8 on Omega
[ordered]   Hello from rank 6 of 8 on Omega
[ordered]   Hello from rank 7 of 8 on Omega
```

Discussion Questions

1. What does the MPI.Environment object do, and what happens if a program uses the communicator before it is created or after it is disposed?

new MPI.Environment(ref args) initialises MPI for the process (it calls MPI_Init): it connects the process to the other ranks started by mpiexec, creates MPI_COMM_WORLD (Communicator.world) and removes any MPI-specific arguments from args. Disposing it at the end of the using block calls MPI_Finalize, which shuts MPI down cleanly. All MPI work must therefore happen inside the using block.

If the program uses the communicator before the environment exists, MPI has not been initialised: Communicator.world is not available and MPI calls fail with an error (in C, calling MPI functions before MPI_Init is an error). After the environment has been disposed, MPI has been finalised and no further MPI calls are allowed; using the communicator then fails, and forgetting to finalise can make mpiexec report that a rank exited abnormally.

2. What are a rank, the communicator size and MPI_COMM_WORLD? How does a process use rank and size to decide what part of the work is its own?

A rank is the ID of a process inside a communicator, from 0 to size - 1. The communicator size is the number of processes in it. MPI_COMM_WORLD (Communicator.world in MPI.NET) is the communicator that contains every process started by mpiexec; in my runs it had size 4, 6 or 8, exactly the number given after -n.

All ranks run the same program, so they use rank and size to choose their own share of the work. For example, with N items each rank takes the block from rank * N / size to (rank + 1) * N / size (as SumCompare does with Scatter), or rank 0 plays a special role (collecting the ordered report, acting as the master in the task farm) while the other ranks do the same worker code.

3. Why did the order of the unordered lines change between runs, while the ordered report was always in rank order? What made the difference?

Each rank is a separate process and prints its unordered line as soon as it gets to it. When each process starts, gets scheduled on a core and reaches the WriteLine is decided by the operating system, and mpiexec forwards the output of each process to the console as it arrives. So the order differed between runs: with 4 ranks it was 1, 0, 2, 3 in the first run and 1, 2, 0, 3 in the second, and with 8 ranks 1, 0, 2, 5, 4, 6, 7, 3 and then 0, 4, 2, 1, 5, 6, 3, 7.

The ordered report was always 0, 1, 2, ... because only rank 0 printed it, and rank 0 explicitly received the lines in rank order with Receive(source = 1, 2, 3, ...). A receive from a specific source waits for that rank's message even if another rank's message arrived first, so the order is fixed by the program, not by timing. The difference is that the order was decided by message matching on one process instead of by the scheduler across many processes.

4. Why do all ranks show the same processor name on your machine, and what would you expect to see on a cluster with several nodes?

All ranks reported the processor name Omega because mpiexec started all of them on my one PC, so every process is running on the same host. MPI.Environment.ProcessorName returns the name of the machine the calling process is running on.

On a cluster with several nodes, the ranks would be spread over the nodes (for example 8 ranks per node), and each rank would print the host name of its own node, such as node01 for ranks 0 to 7 and node02 for ranks 8 to 15. The processor name is a simple way to check where each rank was placed.

5. What is the difference between running the program directly and running it with mpiexec -n 4?

Running HelloMpi.exe directly starts a single process. MPI treats it as a job of one rank (a singleton), so it prints rank 0 of 1 and there is nobody to exchange messages with.

mpiexec -n 4 is the MPI process manager: it starts 4 copies of the program, gives them ranks 0 to 3 in one MPI_COMM_WORLD, sets up the communication channels between them, forwards their console output, and waits for all of them to finish (and aborts the whole job if one of them fails). On a cluster it also places the ranks on the nodes the job was given.

Section 3: Task 2 — Message Passing vs Shared Memory: the Same Sum Two Ways

Source file: SumCompare/Program.cs

```
// Lab 08 - Task 2: Message Passing vs Shared Memory: the Same Sum Two Ways
// The same array of 10,000,000 integers is summed three ways:
//   MPI collective      - Scatter the chunks, every rank sums its chunk, Reduce the partial sums
//   MPI point-to-point  - rank 0 Sends each chunk, every rank sums it and Sends its partial sum back
//   shared memory (TPL) - rank 0 alone uses as many threads as there are ranks, all reading one array
// Each variant is repeated 3 times and the average time is printed by rank 0.
//
// Run:  mpiexec -n 4 .\bin\Release\net48\SumCompare.exe     (ranks must divide 10,000,000)

using System;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;
using MPI;

class Program
{
    const int N = 10_000_000;
    const int Repeats = 3;
    const int DataTag = 1;
    const int ResultTag = 2;

    static Intracommunicator comm = null!;
    static int rank, size;
    static int[] data = null!;                // the full array exists ONLY on rank 0

    static long SumOf(int[] values)
    {
```

```
    long sum = 0;
    for (int i = 0; i < values.Length; i++) sum += values[i];
    return sum;
}

// Rank 0 cuts the array into one chunk per rank.
static int[][] SplitIntoChunks()
{
    int chunkSize = N / size;
    int[][] chunks = new int[size][];
    for (int r = 0; r < size; r++)
    {
        chunks[r] = new int[chunkSize];
        Array.Copy(data, r * chunkSize, chunks[r], 0, chunkSize);
    }
    return chunks;
}

// ---------- variant 1: collective operations ----------
static long CollectiveVariant()
{
    int[][] chunks = rank == 0 ? SplitIntoChunks() : null!;

    int[] myChunk = comm.Scatter<int[]>(chunks, 0);            // every rank receives its own chunk
    long localSum = SumOf(myChunk);                            // work on private memory only
    long total = comm.Reduce<long>(localSum, Operation<long>.Add, 0); // partial sums added at rank 0

    return total;                                              // meaningful on rank 0 only
}

// ---------- variant 2: point-to-point messages ----------
static long PointToPointVariant()
{
    if (rank == 0)
    {
        int[][] chunks = SplitIntoChunks();

        for (int r = 1; r < size; r++)
            comm.Send<int[]>(chunks[r], r, DataTag);           // explicit send of each chunk

        long total = SumOf(chunks[0]);                         // rank 0 sums its own chunk

        for (int r = 1; r < size; r++)
            total += comm.Receive<long>(r, ResultTag);         // explicit receive of each partial sum

        return total;
    }
    else
    {
        int[] myChunk = comm.Receive<int[]>(0, DataTag);
        comm.Send<long>(SumOf(myChunk), 0, ResultTag);
        return 0;
    }
}

// ---------- variant 3: shared memory on rank 0 ----------
static long SharedMemoryVariant()
{
    if (rank != 0) return 0;                                   // other ranks simply wait

    int threads = size;
    int chunkSize = N / threads;
    int[] shared = data;                                       // one array, visible to every thread
    long total = 0;

    Parallel.For(0, threads, new ParallelOptions { MaxDegreeOfParallelism = threads }, t =>
    {
        long local = 0;
        int end = (t + 1) * chunkSize;
        for (int i = t * chunkSize; i < end; i++)
            local += shared[i];                                // every thread reads the SAME array
        Interlocked.Add(ref total, local);
    });
    return total;
}

static void Measure(string name, Func<long> variant, long reference)
```

```
    {
        double totalMs = 0;
        bool correct = true;

        for (int r = 0; r < Repeats; r++)
        {
            comm.Barrier();                         // start all ranks together
            Stopwatch sw = Stopwatch.StartNew();
            long result = variant();
            comm.Barrier();                         // stop when every rank is done
            sw.Stop();

            totalMs += sw.Elapsed.TotalMilliseconds;
            if (rank == 0 && result != reference) correct = false;
        }

        if (rank == 0)
            Console.WriteLine($"{name,-22} avg_time={totalMs / Repeats,10:F2} ms   correct={(correct ? "YES" : "NO")}");
    }

    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            comm = Communicator.world;
            rank = comm.Rank;
            size = comm.Size;

            if (N % size != 0)
            {
                if (rank == 0) Console.WriteLine($"The number of ranks ({size}) must divide {N}. Use 1, 2, 4 or 8.");
                return;
            }

            long reference = 0;
            if (rank == 0)
            {
                Random rng = new Random(2026);
                data = new int[N];
                for (int i = 0; i < N; i++) data[i] = rng.Next(0, 1000);
                reference = SumOf(data);

                Console.WriteLine($"Ranks/threads: {size}  Elements: {N}  Repeats: {Repeats}  " +
                                  $"Processor: {MPI.Environment.ProcessorName}  Logical cores: {System.Environment.ProcessorCount}");
                Console.WriteLine($"Reference sum: {reference}");
            }

            Measure("MPI collective", CollectiveVariant, reference);
            Measure("MPI point-to-point", PointToPointVariant, reference);
            Measure("Shared memory (TPL)", SharedMemoryVariant, reference);
        }
    }
}
```

Timings (average of 3 repetitions, ms)

Ranks / threads	MPI collective	MPI point-to-point	Shared memory (TPL)	All correct? (Y/N)
1	22.18	23.69	11.63	Y
2	60.45	66.27	7.82	Y
4	85.34	82.65	5.59	Y
8	95.83	86.25	6.27	Y

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\SumCompare>mpiexec -n 1 .\bin\Release\net48\SumCompare.exe
Ranks/threads: 1  Elements: 10000000  Repeats: 3  Processor: Omega  Logical cores: 8
Reference sum: 4994589598
MPI collective          avg_time=     22.18 ms   correct=YES
MPI point-to-point      avg_time=     23.69 ms   correct=YES
Shared memory (TPL)     avg_time=     11.63 ms   correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\SumCompare>mpiexec -n 2 .\bin\Release\net48\SumCompare.exe
Ranks/threads: 2  Elements: 10000000  Repeats: 3  Processor: Omega  Logical cores: 8
Reference sum: 4994589598
MPI collective          avg_time=     60.45 ms   correct=YES
MPI point-to-point      avg_time=     66.27 ms   correct=YES
Shared memory (TPL)     avg_time=      7.82 ms   correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\SumCompare>mpiexec -n 4 .\bin\Release\net48\SumCompare.exe
Ranks/threads: 4  Elements: 10000000  Repeats: 3  Processor: Omega  Logical cores: 8
Reference sum: 4994589598
MPI collective          avg_time=     85.34 ms   correct=YES
MPI point-to-point      avg_time=     82.65 ms   correct=YES
Shared memory (TPL)     avg_time=      5.59 ms   correct=YES
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\SumCompare>mpiexec -n 8 .\bin\Release\net48\SumCompare.exe
Ranks/threads: 8  Elements: 10000000  Repeats: 3  Processor: Omega  Logical cores: 8
Reference sum: 4994589598
MPI collective          avg_time=     95.83 ms   correct=YES
MPI point-to-point      avg_time=     86.25 ms   correct=YES
Shared memory (TPL)     avg_time=      6.27 ms   correct=YES
```

Comparison of the two models

Aspect	Shared memory (OpenMP / TPL)	Message passing (MPI)
Memory: who can see which data	All threads share one address space: every thread can read and write the whole array and every shared variable.	Every rank is a separate process with its own private memory. No rank can see another rank's variables; the full array existed only on rank 0.
How a worker gets its data	It simply reads its part of the shared array in place (by index); nothing is copied.	Its chunk must be sent to it in a message (Scatter, or Send/Receive), i.e. copied from rank 0's memory into its own.
How partial results are combined	Each thread adds its partial sum into a shared variable with an atomic operation (Interlocked.Add) or a lock.	Each rank sends its partial sum back in a message: Reduce with Operation<long>.Add, or Send to rank 0 which receives and adds them.
Synchronization	Locks, atomics, barriers and joins on shared variables; races are possible if they are forgotten.	Implicit in the messages: a receive waits until the matching message arrives. No shared data, so no data races between ranks; deadlocks are possible if sends and receives do not match.
Scaling beyond one machine	Limited to the cores and memory of one machine.	Ranks can run on many machines connected by a network; the same program runs on one PC or on thousands of cluster nodes.
Effect of one failed worker	A crashed thread normally brings down the whole process, and all its threads with it.	By default MPI aborts the whole job when one rank fails; the processes are separate, but the computation cannot continue without the lost rank's part.
Main cost seen in this lab	Very small: thread start-up only (5.59 ms with 4 threads).	Copying and serializing the 10,000,000-element array into messages, plus process start-up and message latency; this cost grew with the number of ranks (22 ms at 1 rank to 86-96 ms at 8).

Written comparison

In the shared-memory model of the earlier labs, all threads live in one process and see one address space, so the TPL variant simply let each thread read its part of the same array and combine its partial sum with an atomic add. The message-passing paradigm of MPI is the opposite: every rank is a separate process with private memory, and the only way to share data is to send it explicitly in messages.

That is why SumCompare had to scatter the array from rank 0 to the other ranks and then gather the partial sums back, either with collectives (Scatter and Reduce) or with matching point-to-point Send and Receive calls.

All three variants gave the correct sum at 1, 2, 4 and 8 ranks, but their costs were very different: the shared-memory version took only 5.59 to 11.63 ms, while both MPI variants took 22 to 96 ms and became slower as ranks were added, because copying the 10,000,000 integers into messages costs far more than adding them.

On a single PC, message passing therefore only adds overhead for such a cheap computation. Its value lies elsewhere: because it never relies on shared memory, the same program can run unchanged across many machines connected by a network, where shared memory is not available at all.

Message passing also makes data ownership and synchronization explicit, so there are no data races between ranks, at the price of having to design the communication carefully to avoid deadlocks and to keep the amount of data moved small compared with the computation done on it.

Discussion Questions

1. Why does the MPI version have to scatter the data, while the shared-memory threads simply read the array? What does this tell you about the memory model of MPI?

In MPI each rank is a separate process with its own private memory. The array was created only on rank 0, so the other ranks could not read it at all; the only way for them to get their part is for rank 0 to send it to them in a message (Scatter or Send). The partial sums had to come back in messages too (Reduce or Send/Receive).

The shared-memory threads all run inside one process and share its address space, so they could simply index into the same array. This shows that MPI uses a distributed-memory model: there is no shared data, all data movement is explicit, and every piece of data has an owner rank.

2. Which variant was fastest on your machine, and what extra work do the message-passing variants do? Did adding ranks reduce the time, and why or why not?

The shared-memory (TPL) variant was the fastest at every size: 11.63 ms at 1 thread and 5.59 ms at 4 threads, against 22 to 96 ms for the MPI variants. The message-passing variants additionally have to cut the array into chunks, copy and serialize each chunk into a message, transfer it to another process, deserialize it there, and send the partial sums back, and they synchronize all ranks with barriers.

Adding ranks did not reduce the MPI time; it increased it from 22.18 ms (1 rank) to 95.83 ms (collective, 8 ranks) and 86.25 ms (point-to-point). Summing 10 million integers takes only a few milliseconds, so the work saved by splitting it is far smaller than the cost of moving the data: more ranks means more messages and more copying, while rank 0 still has to create and send all the chunks. The shared-memory version did get faster (11.63 to 5.59 ms at 4 threads), because it moves no data at all; at 8 threads it was slightly slower again (6.27 ms) because thread overhead starts to dominate on 8 logical cores.

3. Describe the semantics of a send and a receive: what must match for a message to be delivered (source, destination, tag, communicator), and what does blocking mean?

A send names the destination rank, a tag and the communicator, and carries the data; a receive names the source rank, a tag and the communicator. A message is delivered to a receive when the communicator is the same, the receive's source is the sender's rank (or anySource), the tag is the same (or anyTag), and the data type matches. Messages between the same pair of ranks with the same tag and communicator are not overtaken: they arrive in the order they were sent.

Blocking means the call does not return until it is safe to continue: a blocking Receive waits until a matching message has arrived and been copied into the receiver's variable; a blocking Send returns when the data has been copied out of the sender's buffer, which for large messages can mean waiting until the receiver has posted its receive. If two ranks both wait to receive from each other first, neither ever sends, and the program deadlocks.

4. Compare the collective and the point-to-point variants. Why are collective operations usually preferred in real programs?

Both variants computed the correct sum and took similar times (for example 85.34 ms collective and 82.65 ms point-to-point at 4 ranks, 95.83 and 86.25 ms at 8). The collective variant expresses the whole pattern in two calls, Scatter and Reduce, in which every rank takes part. The point-to-point variant does the same with a loop of explicit Sends from rank 0, matching Receives on the workers, and a loop of Receives of the partial sums.

Collective operations are usually preferred because they are shorter and clearer, there are no tags or send/receive pairs to get wrong (and so fewer deadlocks), and the MPI library can implement them with efficient algorithms such as tree-based reductions and broadcasts that take log(p) steps instead of p - 1 sequential messages from one rank, using the network topology. On many nodes this makes a large difference, even if it did not on one PC.

5. A computation must run on 1,000 nodes of a cluster. Why is the shared-memory model not an option, and what does the message-passing model offer instead?

The shared-memory model needs all threads to access the same physical memory, and that only exists inside one machine; 1,000 cluster nodes each have their own separate memory and are connected only by a network, so threads on different nodes cannot share an array or a lock. Even the largest single machines have far fewer cores than 1,000 nodes.

The message-passing model was designed for this case: each node runs one or more ranks on its own data, and all cooperation is done with explicit messages over the network. MPI offers a portable way to address every process (ranks in a communicator), point-to-point and collective operations implemented efficiently for the interconnect, and a process manager that starts the job on all the nodes. The same SumCompare program would run on 1,000 nodes with only a larger -n and a host list.

Section 4: Task 3 — Design Note: MPI Deployments and Architecture Styles

Source file: Styles/Program.cs

```
// Lab 08 - Task 3: MPI Deployments and Architecture Styles
// Two small programs that follow two architecture styles:
//   master : task farm (client-server). Rank 0 is the server that hands out tasks;
//            the other ranks are clients that ask for work and return results.
//   ring   : token ring (peer-to-peer). Every rank runs the same code and only
//            talks to its two neighbours; a token travels round the ring.
//
// Run:  mpiexec -n 4 .\bin\Release\net48\Styles.exe master
//       mpiexec -n 4 .\bin\Release\net48\Styles.exe ring

using System;
using System.Diagnostics;
using MPI;

class Program
{
    // ---- task farm settings ----
    const int NumTasks = 200;
    const int RequestTag = 10;              // worker -> master: "I am free, give me work" (carries the worker's rank)
    const int ResultTag = 12;               // worker -> master: the result of the task it just finished
    const int TaskTag = 11;                 // master -> worker: task id, or NoMoreTasks
    const int NoMoreTasks = -1;

    // ---- ring settings ----
    const int Rounds = 1000;
    const int TokenTag = 20;
```

```
  static Intracommunicator comm = null!;
  static int rank, size;

  // The work for one task: tasks have different costs on purpose.
  static long DoTask(int taskId)
  {
     int cost = 200_000 * (1 + taskId % 5);
     long acc = 0;
     for (int k = 0; k < cost; k++) acc += (k ^ taskId) & 15;
     return acc;
  }

  // ================= client-server: task farm =================
  static void TaskFarm()
  {
     if (rank == 0)
     {
        // ---- server (master) ----
        int workers = size - 1;
        int[] served = new int[size];
        bool[] hasOpenTask = new bool[size];         // has this worker got a task whose result is still due?
        int nextTask = 0, finishedWorkers = 0;
        long total = 0;

        Stopwatch sw = Stopwatch.StartNew();
        while (finishedWorkers < workers)
        {
           // a request from ANY worker; the message carries that worker's rank
           int worker = comm.Receive<int>(Communicator.anySource, RequestTag);

           if (hasOpenTask[worker])
              total += comm.Receive<long>(worker, ResultTag);   // collect the result it sent just before

           if (nextTask < NumTasks)
           {
              comm.Send<int>(nextTask, worker, TaskTag);        // serve the next task to this client
              served[worker]++;
              hasOpenTask[worker] = true;
              nextTask++;
           }
           else
           {
              comm.Send<int>(NoMoreTasks, worker, TaskTag);
              hasOpenTask[worker] = false;
              finishedWorkers++;
           }
        }
        sw.Stop();

        long reference = 0;
        for (int t = 0; t < NumTasks; t++) reference += DoTask(t);

        Console.WriteLine($"Task farm (client-server)  ranks={size}  workers={workers}  tasks={NumTasks}
time={sw.Elapsed.TotalMilliseconds:F2} ms");
        for (int w = 1; w < size; w++)
           Console.WriteLine($"  worker {w}: served {served[w]} tasks");
        Console.WriteLine($"correct={(total == reference ? "YES" : "NO")}");
     }
     else
     {
        // ---- client (worker) ----
        while (true)
        {
           comm.Send<int>(rank, 0, RequestTag);                 // ask the server for work
           int taskId = comm.Receive<int>(0, TaskTag);
           if (taskId == NoMoreTasks) break;
           comm.Send<long>(DoTask(taskId), 0, ResultTag);       // return the result ...
        }                                                       // ... and ask again
     }
  }

  // ================= peer-to-peer: token ring =================
  static void TokenRing()
  {
     int left = (rank - 1 + size) % size;
     int right = (rank + 1) % size;
     long token = 0;
```

```
        comm.Barrier();
        Stopwatch sw = Stopwatch.StartNew();

        for (int round = 0; round < Rounds; round++)
        {
            if (rank == 0)
            {
                token++;                            // rank 0 starts each round
                comm.Send<long>(token, right, TokenTag);
                token = comm.Receive<long>(left, TokenTag); // the token comes back round the ring
            }
            else
            {
                token = comm.Receive<long>(left, TokenTag); // same code on every other rank:
                token++;                            // receive from the left neighbour,
                comm.Send<long>(token, right, TokenTag);        // add one, pass to the right neighbour
            }
        }

        sw.Stop();

        if (rank == 0)
        {
            long messages = (long)Rounds * size;
            long expected = messages;                   // every rank adds 1 in every round
            double ms = sw.Elapsed.TotalMilliseconds;
            Console.WriteLine($"Token ring (peer-to-peer)  ranks={size}  rounds={Rounds}  messages={messages}");
            Console.WriteLine($"time={ms:F2} ms  time_per_message={ms / messages:F4} ms");
            Console.WriteLine($"final token={token} expected={expected}  correct={(token == expected ? "YES" : "NO")}");
        }
    }

    static void Main(string[] args)
    {
        using (new MPI.Environment(ref args))
        {
            comm = Communicator.world;
            rank = comm.Rank;
            size = comm.Size;
            string mode = args.Length > 0 ? args[0] : "master";

            if (size < 2)
            {
                if (rank == 0) Console.WriteLine("Run with at least 2 ranks, e.g. mpiexec -n 4 ...");
                return;
            }

            if (mode == "ring") TokenRing();
            else TaskFarm();
        }
    }
}
```

Task farm (client-server style)

Ranks	Workers	Tasks served per worker	Correct? (Y/N)
4	3	Run 1: 74 / 72 / 54; run 2: 78 / 59 / 63 (200 tasks)	Y
8	7	31 / 32 / 25 / 26 / 26 / 30 / 30 (200 tasks)	Y

Token ring (peer-to-peer style)

Ranks	Time (ms)	Time per message (ms)	Correct? (Y/N)
4	15.83	0.0040	Y
8	25.12	0.0031	Y

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\Styles>mpiexec -n 4 .\bin\Release\net48\Styles.exe master
mpiexec -n 8 .\bin\Release\net48\Styles.exe master
mpiexec -n 4 .\bin\Release\net48\Styles.exe ring
mpiexec -n 8 .\bin\Release\net48\Styles.exe ringTask farm (client-server)  ranks=4  workers=3  tasks=200  time=40.04 ms
  worker 1: served 74 tasks
  worker 2: served 72 tasks
  worker 3: served 54 tasks
correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\Styles>mpiexec -n 4 .\bin\Release\net48\Styles.exe master
Task farm (client-server)  ranks=4  workers=3  tasks=200  time=33.95 ms
  worker 1: served 78 tasks
  worker 2: served 59 tasks
  worker 3: served 63 tasks
correct=YES
```

```
C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\Styles>mpiexec -n 8 .\bin\Release\net48\Styles.exe master
Task farm (client-server)  ranks=8  workers=7  tasks=200  time=36.89 ms
  worker 1: served 31 tasks
  worker 2: served 32 tasks
  worker 3: served 25 tasks
  worker 4: served 26 tasks
  worker 5: served 26 tasks
  worker 6: served 30 tasks
  worker 7: served 30 tasks
correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\Styles>mpiexec -n 4 .\bin\Release\net48\Styles.exe ring
Token ring (peer-to-peer)  ranks=4  rounds=1000  messages=4000
time=15.83 ms  time_per_message=0.0040 ms
final token=4000 expected=4000  correct=YES

C:\Users\DELL\Desktop\BSCS-ev-7-C\PDC\PDC_Repo\Lab8_PDC\Styles>mpiexec -n 8 .\bin\Release\net48\Styles.exe ring
Token ring (peer-to-peer)  ranks=8  rounds=1000  messages=8000
time=25.12 ms  time_per_message=0.0031 ms
final token=8000 expected=8000  correct=YES
```

Architecture styles compared for an MPI cluster

Style	How the MPI programs / cluster map onto it	Strengths	Weaknesses	Fit (1-5)
Client-server	The task farm: rank 0 is the server that owns the task list and serves tasks on request; ranks 1..n-1 are clients that ask for work and return results. In a deployment, the login node and job scheduler also act as servers for users.	Simple to write and reason about; automatic load balancing (fast workers get more tasks); central place for results and bookkeeping.	Rank 0 is a bottleneck as workers grow (every request goes through it) and a single point of failure; workers sit idle while waiting for the server.	4
Peer-to-peer	The token ring and most SPMD code: every rank runs the same program and exchanges data directly with its neighbours (halo exchange, ring, all-to-all); no rank is in charge.	No central bottleneck; communication is spread over all links, so it scales to many nodes; symmetric code.	Harder to balance uneven work; the slowest peer or link sets the speed; a ring is sequential (the token visits one rank at a time).	4
Layered / tiered	The cluster as a stack: user and login node, job scheduler/resource manager, mpiexec and the MPI runtime, the MPI library and application, then the OS and the network interconnect. Each layer only uses the one below.	Separation of concerns; parts can be replaced independently (another scheduler, another network) without changing the program; good for management and security.	Each layer adds overhead and latency; it describes the system's organisation, not how the ranks of one computation talk to each other.	5

Design note

1. Deployment

I consider a small MPI cluster of 16 compute nodes, each with 8 cores and its own memory, connected by a high-speed interconnect (for example InfiniBand or 10 Gb Ethernet). Users log in to a separate login node, submit jobs to a job scheduler (for example Slurm, or the HPC Pack scheduler on Windows), and the scheduler decides which nodes a job gets. mpiexec then starts the ranks, typically one rank per core, for example 8 ranks per node and 128 ranks for a full-cluster job, all in one MPI_COMM_WORLD. On my PC the same programs ran as 4 to 8 ranks on one machine (processor name Omega), which is the one-node case of this deployment.

2. Client-server mapping

The task farm in Styles is client-server: rank 0 is the server that owns the task list, and the workers are clients that send a request, receive a task, and return the result. With 3 workers the 200 tasks were split 74 / 72 / 54, and with 7 workers 31 / 32 / 25 / 26 / 26 / 30 / 30, so the server balanced the work automatically according to how fast each client came back. At the deployment level the login node and the scheduler are also servers that clients (users and their jobs) send requests to. The weakness is that everything passes through the server: with hundreds of workers rank 0 would have to answer every request alone, and if it fails the whole farm stops.

3. Peer-to-peer mapping

The token ring is peer-to-peer: every rank runs exactly the same code and talks only to its left and right neighbours, and no rank is in charge of the others. Most real MPI programs are SPMD and follow this style: domain decomposition with halo exchange, all-reduce and all-to-all exchanges all treat the ranks as equal peers. My ring passed 4,000 messages in 15.83 ms with 4 ranks and 8,000 messages in 25.12 ms with 8 ranks, about 3 to 4 microseconds per message, which shows how cheap a short message is on one node. Because the token is only on one rank at a time, a ring is sequential, so its speed is limited by the latency of each hop and by the slowest peer.

4. Layered / tiered mapping

The cluster as a whole is layered. From the top: the user and the login node (access tier); the job scheduler and resource manager (management tier); mpiexec, the process manager and the MPI runtime that start and connect the ranks; the MPI library (MPI.NET on MS-MPI here) with the application on top of it; and at the bottom the operating system and the network interconnect. Each layer uses only the services of the layer below, so a different scheduler or network can be used without changing the application.

5. Comparison

Client-server is simple and balances load well but centralises traffic and failure; peer-to-peer removes the central bottleneck and spreads communication over all links, which is what large computations need, but is harder to balance when the work is uneven; layered describes how the whole cluster is organised and managed rather than how the ranks of one job talk to each other. The three styles are therefore not alternatives at the same level: a real MPI cluster uses a layered organisation, a client-server front end (login node and scheduler), and mostly peer-to-peer communication inside each job.

6. Recommendation

For the MPI cluster as a whole I recommend a layered/tiered architecture, with peer-to-peer (SPMD) communication between the ranks of each job. Layering keeps users, scheduling, the MPI runtime and the network separate, which makes the cluster manageable, shareable between users and easy to upgrade. Peer-to-peer communication inside the job scales to thousands of ranks because there is no single rank through which all data must pass. Client-server remains the right choice for part of the system: the login node and scheduler, and inside a job a master-worker task farm when the tasks are independent and of unpredictable cost, as long as the number of workers is moderate or a hierarchy of masters is used.

Discussion Questions

1. Which of your programs follows the client-server style, which roles do rank 0 and the other ranks play, and where could a bottleneck appear as the number of workers grows?

The task farm (Styles master) follows the client-server style. Rank 0 is the server (master): it owns the list of 200 tasks, waits for requests from any worker, collects each finished result, and replies with the next task id or a stop signal. Ranks 1 to n-1 are clients (workers): they ask for work, compute the task and return the result. The work was balanced automatically: 74 / 72 / 54 tasks with 3 workers and 25 to 32 tasks each with 7 workers.

The bottleneck is rank 0. Every request and every result passes through it, and it handles them one at a time, so as the number of workers grows, workers spend more time waiting for the server to answer; with very short tasks or hundreds of workers the server cannot keep up. Rank 0 is also a single point of failure. The usual fixes are larger tasks, giving each worker several tasks at a time, or a hierarchy of sub-masters.

2. Which program follows the peer-to-peer style? What is symmetric about it, and what limits its speed?

The token ring (Styles ring) follows the peer-to-peer style. Every rank except rank 0 runs exactly the same code: receive the token from the left neighbour, add one, send it to the right neighbour. Each rank talks only to its two neighbours and none of them controls the others, so the structure is symmetric; rank 0's only special role is starting each round.

Its speed is limited by the latency of each hop, because the token is on only one rank at a time, so the ring is completely sequential: 4,000 messages took 15.83 ms with 4 ranks and 8,000 took 25.12 ms with 8 ranks, about 3 to 4 microseconds per message. More ranks means a longer ring and more hops per round, and the slowest rank or link holds up everyone.

3. Describe a layered or tiered view of an MPI cluster, from the user down to the network. What does each layer or tier do?

From the user down to the network:

1. Access tier: the user connects to a login node, edits and compiles programs and submits jobs.

2. Management tier: the job scheduler and resource manager (for example Slurm or HPC Pack) queues jobs, decides which nodes each job gets, and starts and monitors it.

3. Runtime layer: mpiexec and the MPI process manager start the ranks on the allocated nodes, assign rank numbers and set up communication between them.

4. Library and application layer: the program (HelloMpi, SumCompare, Styles) uses the MPI library (MPI.NET on top of MS-MPI) for sends, receives and collectives.

5. System and network layer: the operating system on each node, and the interconnect (Ethernet or InfiniBand, or shared memory between ranks on the same node) that actually moves the bytes.

Each layer offers services to the layer above and hides the details of the layers below, so for example the program does not change when the network or the scheduler changes.

4. Which style best fits an MPI cluster, and why? Give at least two reasons, and say when a different style would be better for part of the system.

A layered/tiered organisation of the cluster with peer-to-peer communication inside each job fits best. Reasons: (1) layering separates users, scheduling, the MPI runtime and the hardware, which makes the cluster manageable, secure and shareable between many users and jobs, and lets each part be replaced independently; (2) peer-to-peer (SPMD) communication has no central rank that all data must pass through, so it scales to thousands of ranks and uses all network links in parallel; (3) MPI collectives are themselves implemented as peer-to-peer patterns (trees and rings) between equal ranks.

A different style is better for parts of the system: the login node and scheduler are naturally client-server, and inside a job a master-worker (client-server) task farm is the better choice when tasks are independent and of unpredictable cost, as in my task farm, where it balanced 200 tasks of different cost automatically.

5. What happens in each style if one node fails? What does MPI do by default when one rank fails, and what does that mean for the choice of style?

Client-server: if a worker (client) fails, the server can in principle give its task to another worker; if the server fails, everything stops, because it is a single point of failure. Peer-to-peer: there is no central point, but each peer owns part of the data or is part of the ring, so the loss of one peer breaks the ring or loses that part of the computation unless the data is replicated. Layered: a failure is contained in one layer; for example the scheduler can mark a node as down and run later jobs elsewhere, but a running job on that node is still lost.

By default MPI treats the failure of one rank as fatal for the whole job: the default error handler (MPI_ERRORS_ARE_FATAL) aborts all the ranks, and mpiexec kills the remaining processes. So, within a standard MPI job, the choice of style does not by itself give fault tolerance. Long computations instead rely on checkpoint/restart through the scheduler, and if a style must survive the loss of a node, the fault tolerance has to be designed in explicitly, which is easier in a client-server task farm (re-issue lost tasks) than in a tightly coupled peer-to-peer computation.

Section 5: GitHub Repository

All source code for this lab is pushed to the lab8 branch of my repository:

https://github.com/fahad1084/-PDC_Labs/tree/lab8